

Homework



Homework

- In today's homework, you will continue on the **ToDo List** app using React.
- You will continue use **Code Sandbox** for this homework.
- Please fork the app (from last homework) in Sandbox before you begin this homework. Modify the name to include **v2** (as in version 2).
- In the last homework, you'd:
 - Created an app functional component in App.js that returns some JSX elements
 - Created two functional components to contain todos list data
 - Created a date and array to store days
 - Created a conditional statement to test the current day so the correct todos list will be pulled
 - Used Reactstrap and Bootstrap to format and style JSX elements
 - Used minimal CSS to style JSX elements



Homework

- In this homework, you will start by converting two existing scripts:
 - App functional component to a class component. Reason for this is, class can hold/store data in one place and pass them to child components.
 - The existing functional components that hold their respective data will be dismantled. The data will all be moved to one place.
- To convert to a class component, first import React components from the React library (React is already a dependency by default in Codesandbox):

```
import "./styles.css";  
import React, { Component } from "react";  
import { ListGroup, ListGroupItem } from "reactstrap";  
import "bootstrap/dist/css/bootstrap.min.css";
```



Homework

- Convert the app component to a class:

```
export default function App() {  
  let d = new Date(); /* Get current date */  
}
```



```
class App extends Component {  
  let d = new Date(); /* Get current date */  
}
```

- Scroll to the bottom of the page and add the export here:

```
    </ListGroup>  
  );  
}  
  
export default App;
```



Homework

- Unlike functional components, anything that contains within class components needs to be rendered, except for the data that it is stored. Render() is a class component method and is the only required method in a class. All other methods are optional. Wrap the following within the render() method:

```
class App extends Component {  
  render() {  
    let d = new Date(); /* Get current date */  
  }  
}
```

```
    {todo}  
  </div>  
)  
);  
}
```



Homework

- Now that you have converted the app component to a class, the next step is to move the data out from the functional components to the app class, as states. To be able to hold the data as states, you need the JavaScript constructor() method. Constructor allows you to declare properties for use within the class component whether by inheritance or that you create right within the constructor.

```
class App extends Component {  
  constructor(props) {  
    super(props);  
  }  
  render() {
```

- The super(props) method will inherit any properties from the parent, in this case the React component. The constructor takes those inherited props (and any newly created ones that are stored as states) so that they are accessible by this component.



Homework

- Next, you will move the todos list within the constructor as states. But first set up the states as follows:
- Now, add in the data and at the same time, you will also add additional properties:

```
constructor(props) {  
  super(props);  
  this.state = {  
    }  
}
```

```
this.state = {  
  TodoList1: [  
    { id: 1, text: "Eat Breakfast", done: false },  
    { id: 2, text: "Walk Dog", done: false },  
    { id: 3, text: "Take Shower", done: false }  
  ],  
  TodoList2: [  
    { id: 1, text: "Get Lunch", done: false },  
    { id: 2, text: "Run a Mile", done: false },  
    { id: 3, text: "Take Bath", done: false }  
  ]  
};
```



Homework

- With all data moved, you will dismantle the functional components that used to have the data, namely TodoList1, TodoList2, etc. Remove these functions.
- As a replacement, you will create a new function with the basic structures that resembles those you removed. The name of the function will be more generic - Todos:

```
function Todos() {  
  return (  
    <ListGroup>  
      <ListGroupItem  
        tag="a"  
        href="#"  
        className="align-self-center py-2 w-50 bg-info text-white"  
      >  
      </ListGroupItem>  
    </ListGroup>  
  );  
}
```



Homework

- Let's leave this new function component for a moment and modify the current calling of the functions within the render() in the class. Instead of calling different functions based on different conditions, it will now call this new function by passing to it the correct data from the state via an attribute so that the function will receive the data:

```
if (n === "Monday") {  
  todo = <Todos todolists={this.state.TODOList1} />;  
} else if (n === "Tuesday") {  
  todo = <Todos todolists={this.state.TODOList2} />;  
}
```

Note: the same function being called will have the same attribute name but with different state data.



Homework

- Okay with the function calling now modified, let's return the new function.
- If this new function receives data from another component (in this example, the App class), in React, this function is perceived as the child component of App. When the parent component App pass it's state data to a child component, these data are passed as props not state.
- So therefore, in the function, insert the parameter **props** within the function's parenthesis:

```
function Todos(props) {  
  return (  
    <ListGroup>  
      <ListGroupItem
```



Homework

- So with the props passed into the function, you want to disperse this group of todolist data that contain 3 todo items into item #1, #2, #3. The array map method ie: .map() will help you do that. It loops through an array of items.
- Remember, your new function now contains only one JSX list item – so you will run the map method right before it:

Note:

In the map method, you return a single object and that object name can be anything. Here, it's named todolist (singular vs plural is commonly used). This object will be used to extract properties.

```
function Todos(props) {  
  return (  
    <ListGroup>  
      {props.todolists.map((todolist) => (  
        <ListGroupItem  
          tag="a"  
          href="#"  
          className="align-self-center py-2 w-50 bg-info text-white"  
        >  
          </ListGroupItem>  
        )}}  
    </ListGroup>  
  )
```



Homework

- Even though you have mapped each item from the `todolist` array, at this time those items are not dispersed out within the JSX to be output. You do this next.
- The first thing is to insert a `key` to the highest JSX element after mapping. This key is essential for the `map` method so as to give each item an identification. The most common and logical info from the array to use as key is the `id`.

```
function Todos(props) {  
  return (  
    <ListGroup>  
      {props.todolists.map((todolist) => (  
        <ListGroupItem  
          key={todolist.id}  
          tag="a"  
          href="#"
```



Homework

- The current list does not output any information from the array items. This is because you'd transferred out the static data to the state. Now it's time to bring them back not as static text but dynamically like how the id is extracted for the key:

```
href="#"
className="align-
>
{todolist.text}
</ListGroupItem>
```

- At this time, you should be able to see the items display again on the browser.



Homework

- Your job is done. You have successfully moved the data out from individual functions to come from a ***single source of truth*** ie. all in one place called state, used the array map method to extract each array item and finally dispersed out into JSX elements.
- One thing worth noting is when moving data out from child components (in this case the functional components) to a parent component (in this case the App class component), in React, that process is called ***Lifting State Up***.
- The main reason to lift state up is to make data available to all child components and in this case, App acts as the main component that holds the *single source of truth* for functional components to access and use.
- However, you are not quite done yet. You will add this extra feature on the app next.



Homework

This extra feature works like this:

- When user clicks on one of the todos item, an extra text note will be displayed below the list. First, let's add notes for each todos item in the array:

```
this.state = {
  TodoList1: [
    {
      id: 1,
      text: "Eat Breakfast",
      note: "Make Eggs and Pancakes",
      done: false
    },
    { id: 2,
      text: "Walk Dog",
      note: "Bring doggie bag",
      done: false
    },
    {
      id: 3,
      text: "Take Shower",
      note: "Shower no later than 10am!",
      done: false
    }
  ]
},
```

```
TodoList2: [
  { id: 1,
    text: "Get Lunch",
    note: "Lunch at Sandbox",
    done: false
  },
  {
    id: 2,
    text: "Run a Mile",
    note: "Stop by mom at end of run",
    done: false
  },
  {
    id: 3,
    text: "Take Bath",
    note: "Prep half hour before. Replenish bath oil.",
    done: false
  }
]
```



Homework

- Let's convert the list item in JSX from a hypertext link to behave as a **button**. And then remove the href and replace with an **onClick** attribute in the element:

```
<ListGroupItem
  key={todolist.id}
  tag="button"
  onClick={}
  className="align-s
>
  {todolist.text}
</ListGroupItem>
```

- In Javascript, the onClick attribute typically calls a function/method to handle the event and at same time could pass data to the function/method or retrieve input data from the element. This function/method is refer to as the event handler.



Homework

- But let's create this event handler first. The handler should be placed within the same functional component but outside of the return. An arrow function will be used, although a regular function will also work:

```
function Todos(props) {  
  handleNote = () => {  
    ;  
  };  
  return (  
    <ListGroup>
```

- You can now call this handler through the attribute *onClick*:

```
tag="button"  
onClick={() => this.handleNote(todolist.note)}  
className="align-self-center py-2 w-50 bg-info"
```

Note: arrow function is used to call an event handler in a functional component, and passing data to the handler.



Homework

- With data being passed to the handler, a parameter will be needed to receive it:

```
function Todos(props) {  
  handleNote = (getNote) => {  
  
  };  
}
```

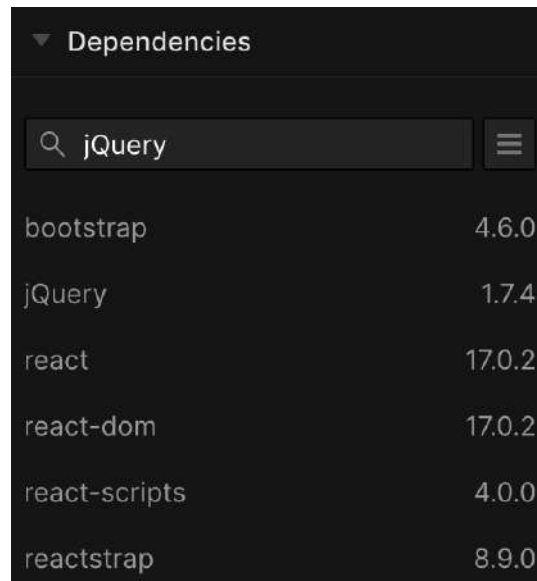
- Next, let's add a JSX element to display this note data, which will be displayed below the list:

```
    </ListGroupItem>  
  )})  
  <div id="note" className="mt-5 font-weight-bold text-dark"></div>  
</ListGroup>
```



Homework

- To display the note data in this JSX element, Javascript's innerHTML or jQuery's append will be used. In this case, jQuery will be opted.
- To use jQuery in React, you will need to install the package. In Codesandbox, add this in the dependencies:



▼ Dependencies	
jQuery	
bootstrap	4.6.0
jQuery	1.7.4
react	17.0.2
react-dom	17.0.2
react-scripts	4.0.0
reactstrap	8.9.0



Homework

- Next, import into the app:

```
import "./styles.css";
import React, { Component } from "react";
import { ListGroup, ListGroupItem } from "reactstrap";
import "bootstrap/dist/css/bootstrap.min.css";
import $ from "jQuery";
```

- The final piece of the puzzle is to display the note data in the JSX element:

```
function Todos(props) {
  handleNote = (getNote) => {
    $("#note").empty().append(getNote);
  };
  return (
```

Note: the empty() method is chained here to ensure the previous note data is cleared before a new note is displayed.



Homework

- Give it a test drive on the browser:

Monday's

Todo List

Eat Breakfast
Walk Dog
Take Shower

Shower no later than 10am!

or

Tuesday's

Todo List

Get Lunch
Run a Mile
Take Bath

Stop by mom at end of run

Note:

Feel free to inspect elements on your browser to view the html output.



Homework

- Enhancements
 - Remove note data via a button
 - Style the note data display
- Wrap note display with a second JSX list item element. And change the margin top of the div element to size 1:

```
    )})  
    <ListGroupItem  
      id="notebox"  
      className="border border-dark rounded m-5"  
    >  
      <div id="note" className="mt-1 font-weight-bold text-dark"></div>  
    </ListGroupItem>  
  </ListGroup>
```



Homework

- Add a JSX button element within the new note display box:

```
<div id="note" className="mt-1 font-weight-bold text-dark"></div>  
<button  
  id="btn"  
  className="w-25 mt-2 align-self-center bg-warning border-0"  
  onClick={() => this.dismissNote()}  
>  
  Done  
</button>  
</ListGroupItem>
```

- Hide the note display box by default. Add this to [styles.css](#):

```
ul #notebox {  
  visibility: hidden;  
}
```



Homework

- Add a jQuery script in the handler to make the note display box visible:

```
handleNote = (getNote) => {  
  $("#note").empty().append(getNote);  
  $("#notebox").css("visibility", "visible");  
};
```

- Add the handler to dismiss/close the note display box:

```
handleNote = (getNote) => {  
  $("#note").empty().append(getNote);  
  $("#notebox").css("visibility", "visible");  
};  
  
dismissNote = () => {  
  $("#note").empty();  
  $("#notebox").css("visibility", "hidden");  
};
```



Homework

- On the browser preview:

Monday's Todo List

Eat Breakfast
Walk Dog
Take Shower

Bring doggie bag

Done

or

Tuesday's Todo List

Get Lunch
Run a Mile
Take Bath

Lunch at Sandbox

Done

Note:

Feel free to inspect elements on your browser to view the html output.



Homework

- **DUE:**

On or before this Sunday, 10.30PM PT.

- **Submission:**

Post Code Sandbox URL on GAP Week 1 Day 2 Homework



Connect with Us (WEB403/603/803)

Professor

Rich Loke

richloke@westcliff.edu

Teaching
Assistant

Elizabeth Kipp

elizabethkipp@westcliff.edu

Teaching
Assistant

Erin Tustin

erintustin@westcliff.edu

End of Presentation